

Отчёт по лабораторной работе №4

Базовая настройка HTTP-сервера Apache

Цыкунова Екатерина Михайловна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	19
4	Контрольные вопросы	20

Список иллюстраций

2.1	Установка группы пакетов Basic Web Server	7
2.2	Просмотр конфигурационных файлов, настройка firewalld и за- пуск Apache	8
2.3	Проверка доступности HTTP-сервера с виртуальной машины client	9
2.4	Анализ журналов error_log и access_log HTTP-сервера	10
2.5	Добавление записи www в прямую DNS-зону emcikunova.net . . .	11
2.6	Добавление PTR-записи www.emcikunova.net в обратную DNS-зону	12
2.7	Конфигурация виртуального хоста server.emcikunova.net	13
2.8	Конфигурация виртуального хоста www.emcikunova.net	13
2.9	Тестовая страница виртуального хоста server.emcikunova.net . . .	14
2.10	Тестовая страница виртуального хоста www.emcikunova.net . . .	14
2.11	Настройка прав, восстановление контекстов SELinux и переза- пуск Apache	15
2.12	Проверка виртуального хоста server.emcikunova.net с машины client	15
2.13	Проверка виртуального хоста www.emcikunova.net с машины client	16
2.14	Сохранение конфигурации HTTP и DNS в структуре provision . . .	17
2.15	Provisioning-скрипт http.sh для автоматической настройки HTTP-сервера	18

Список таблиц

1 Цель работы

Приобретение практических навыков по установке и базовому конфигурированию HTTP-сервера Apache.

2 Выполнение лабораторной работы

На виртуальной машине `server` перехожу в режим суперпользователя и устанавливаю набор пакетов, необходимый для работы HTTP-сервера Apache, командой `dnf -y groupinstall "Basic Web Server"` (см. рис. Рисунок 2.1).

В процессе установки менеджер пакетов выполняет проверку транзакции и устанавливает веб-сервер `httpd`, пакет `httpd-core` с основными компонентами Apache, набор служебных утилит `httpd-tools`, файловую структуру `httpd-filesystem`, документацию `httpd-manual`, а также дополнительные модули `mod_http2`, `mod_lua`, `mod_ssl` и другие зависимости. Наличие `mod_ssl` позволяет в дальнейшем использовать HTTPS, а `mod_http2` добавляет поддержку протокола HTTP/2.

Строка `Transaction test succeeded` свидетельствует об успешной предварительной проверке транзакции, а сообщение `Complete!` подтверждает завершение установки без ошибок. В результате на виртуальной машине установлен HTTP-сервер Apache версии 2.4 и необходимые для его функционирования компоненты.

```

Transaction test succeeded.
Running transaction
  Preparing : 1/1
  Running scriptlet: httpd filesystem-2.4.63-13.el10_2.6.noarch 1/14
  Installing : httpd filesystem-2.4.63-13.el10_2.6.noarch 1/14
  Installing : apr-1.7.5-3.el10.x86_64 2/14
  Installing : apr-util-lmdb-1.6.3-23.el10_2.1.x86_64 3/14
  Installing : apr-util-openssl-1.6.3-23.el10_2.1.x86_64 4/14
  Installing : apr-util-1.6.3-23.el10_2.1.x86_64 5/14
  Installing : httpd-tools-2.4.63-13.el10_2.6.x86_64 6/14
  Installing : httpd-core-2.4.63-13.el10_2.6.x86_64 7/14
  Installing : mod_http2-2.0.29-4.el10_2.2.x86_64 8/14
  Installing : mod_lua-2.4.63-13.el10_2.6.x86_64 9/14
  Installing : rocky-logos-httpd-100.5-3.el10.noarch 10/14
  Installing : httpd-2.4.63-13.el10_2.6.x86_64 11/14
  Running scriptlet: httpd-2.4.63-13.el10_2.6.x86_64 11/14
  Installing : httpd-manual-2.4.63-13.el10_2.6.noarch 12/14
  Installing : mod_fcgid-2.3.9-35.el10.x86_64 13/14
  Installing : mod_ssl-1:2.4.63-13.el10_2.6.x86_64 14/14
  Running scriptlet: httpd-2.4.63-13.el10_2.6.x86_64 14/14
  Running scriptlet: mod_ssl-1:2.4.63-13.el10_2.6.x86_64 14/14

Installed:
  apr-1.7.5-3.el10.x86_64
  apr-util-lmdb-1.6.3-23.el10_2.1.x86_64
  httpd-2.4.63-13.el10_2.6.x86_64
  httpd filesystem-2.4.63-13.el10_2.6.noarch
  httpd-tools-2.4.63-13.el10_2.6.x86_64
  mod_http2-2.0.29-4.el10_2.2.x86_64
  mod_lua-2.4.63-13.el10_2.6.x86_64
  mod_ssl-1:2.4.63-13.el10_2.6.x86_64
  apr-util-1.6.3-23.el10_2.1.x86_64
  apr-util-openssl-1.6.3-23.el10_2.1.x86_64
  httpd-core-2.4.63-13.el10_2.6.x86_64
  httpd-manual-2.4.63-13.el10_2.6.noarch
  mod_fcgid-2.3.9-35.el10.x86_64
  mod_lua-2.4.63-13.el10_2.6.x86_64
  rocky-logos-httpd-100.5-3.el10.noarch

Complete!
[root@server.emcikunova.net ~]#

```

Рисунок 2.1: Установка группы пакетов Basic Web Server

После установки просматриваю содержимое основных каталогов конфигурации Apache командами `ls /etc/httpd/conf` и `ls /etc/httpd/conf.d` (см. рис. Рисунок 2.2).

В каталоге `/etc/httpd/conf` находятся основной конфигурационный файл `httpd.conf` и файл `magic`. Файл `httpd.conf` содержит глобальные настройки веб-сервера: параметры прослушиваемых портов, подключение модулей, расположение каталогов с веб-контентом, правила доступа, параметры журналирования и другие основные директивы. Файл `magic` используется модулем определения типов содержимого файлов.

Каталог `/etc/httpd/conf.d` предназначен для дополнительных конфигурационных файлов. В нём присутствуют `autoindex.conf`, `fcgid.conf`, `manual.conf`, `ssl.conf`, `userdir.conf`, `welcome.conf` и другие файлы. `ssl.conf` содержит настройки HTTPS, `userdir.conf` управляет доступом к пользовательским веб-каталогам, `welcome.conf` отвечает за стандартную тестовую страницу Apache при отсутствии пользовательского содержимого, а `autoindex.conf` содержит настройки отображения содержимого каталогов.

Для разрешения входящих HTTP-соединений добавляю службу http в конфигурацию межсетевого экрана. Команда `firewall-cmd --add-service=http` применяет разрешение к текущей конфигурации `firewalld`, а `firewall-cmd --add-service=http --permanent` сохраняет его после перезагрузки системы. В обоих случаях получено сообщение `success`.

После настройки межсетевого экрана разрешаю автоматический запуск Apache командой `systemctl enable httpd`. В результате создаётся символическая ссылка для запуска `httpd.service` при переходе системы в целевой режим `multi-user.target`. Затем запускаю веб-сервер командой `systemctl start httpd`.

```
[root@server.emcikunova.net ~]#  
[root@server.emcikunova.net ~]# ls /etc/httpd/conf  
httpd.conf  magic  
[root@server.emcikunova.net ~]# ls /etc/httpd/conf.d/  
autoindex.conf  fcgid.conf  manual.conf  README  ssl.conf  userdir.conf  welcome.conf  
[root@server.emcikunova.net ~]# firewall-cmd --add-service=http  
success  
[root@server.emcikunova.net ~]# firewall-cmd --add-service=http --permanent  
success  
[root@server.emcikunova.net ~]# systemctl enable httpd  
Created symlink '/etc/systemd/system/multi-user.target.wants/httpd.service' -> '/usr/lib/systemd/system/httpd.service'.  
[root@server.emcikunova.net ~]# systemctl start httpd  
[root@server.emcikunova.net ~]#
```

Рисунок 2.2: Просмотр конфигурационных файлов, настройка `firewalld` и запуск Apache

Для проверки базовой конфигурации запускаю виртуальную машину `client` и в веб-браузере обращаюсь к серверу по адресу `http://192.168.1.1` (см. рис. Рисунок 2.3).

В браузере отображается стандартная страница HTTP `Server Test Page` для Rocky Linux. Её появление подтверждает, что служба `httpd` успешно запущена, принимает соединения на TCP-порту 80 и доступна с клиентской виртуальной машины через внутреннюю сеть. Кроме того, прохождение запроса подтверждает корректность правила `firewalld`, разрешающего службу `http`.

Браузер отображает пометку `Not Secure`, поскольку соединение на данном этапе выполняется по обычному протоколу HTTP и не защищено TLS.

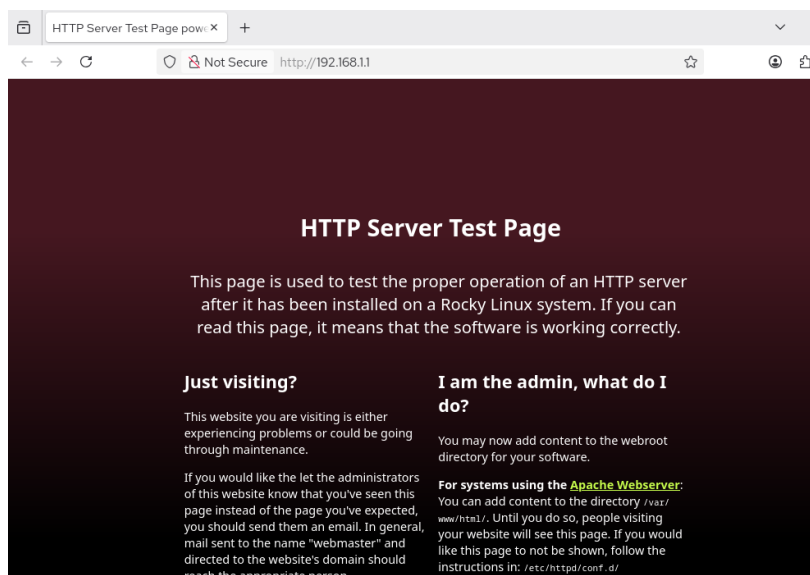


Рисунок 2.3: Проверка доступности HTTP-сервера с виртуальной машины client

На виртуальной машине server анализирую журналы работы Apache. Для просмотра диагностических сообщений использую команду `tail -f /var/log/httpd/error_log`, а для наблюдения за запросами клиентов — `tail -f /var/log/httpd/access_log` (см. рис. Рисунок 2.4).

В журнале ошибок присутствуют сообщения о включении механизма suEXEC, политике SELinux и запуске Apache. Строка `Apache/2.4.63 (Rocky Linux) ... configured -- resuming normal operations` свидетельствует о том, что сервер успешно прочитал конфигурацию и перешёл к штатной работе. В журнале также отображается команда запуска `/usr/sbin/httpd -D FOREGROUND`.

В `access_log` фиксируются обращения от узла 192.168.1.30, который является клиентом внутренней сети. В журнале присутствуют запросы GET к корневому ресурсу `/` и вспомогательным файлам тестовой страницы. Для части ресурсов сервер возвращает код 200, означающий успешную обработку запроса. Начальный запрос к `/` также фиксируется с кодом 403, поскольку стандартная конфигурация Rocky Linux при пустом каталоге `/var/www/html` использует механизм тестовой страницы вместо обычного списка содержимого каталога.

Запрос к favicon.ico завершается кодом 404, поскольку данный файл отсутствует. Это не препятствует работе основного веб-сайта.

По строкам журнала также видно, что клиент использует браузер Firefox. Следовательно, запрос из браузера виртуальной машины client действительно поступает на Apache и обрабатывается сервером.

```
liov@server.emcikunova.net ~]$  
[root@server.emcikunova.net ~]# tail -f /var/log/httpd/error_log  
[Sat Sep 19 11:07:35.764306 2026] [suexec:notice] [pid 15984:tid 15984] AH01232: suEXEC mechanism enable  
d (wrapper: /usr/sbin/suexec)  
[Sat Sep 19 11:07:35.773732 2026] [lbmethod_heartbeat:notice] [pid 15984:tid 15984] AH02282: No slotmem  
from mod_heartbeat  
[Sat Sep 19 11:07:35.775615 2026] [systemd:notice] [pid 15984:tid 15984] SELinux policy enabled; httpd r  
unning as context system_u:system_r:httpd_t:s0  
[Sat Sep 19 11:07:35.778780 2026] [mpm_event:notice] [pid 15984:tid 15984] AH00489: Apache/2.4.63 (Rocky  
Linux) OpenSSL/3.5.5 mod_fcgid/2.3.9 configured -- resuming normal operations  
[Sat Sep 19 11:07:35.778792 2026] [core:notice] [pid 15984:tid 15984] AH00094: Command line: '/usr/sbin/  
httpd -D FOREGROUND'  
^C  
[root@server.emcikunova.net ~]#  
[root@server.emcikunova.net ~]# tail -f /var/log/httpd/access_log  
192.168.1.30 - - [19/Sep/2026:11:09:27 +0000] "GET / HTTP/1.1" 403 7620 "-" "Mozilla/5.0 (X11; Linux x86  
_64; rv:140.0) Gecko/20100101 Firefox/140.0"  
192.168.1.30 - - [19/Sep/2026:11:09:27 +0000] "GET /icons/poweredby.png HTTP/1.1" 200 15443 "http://192.  
168.1.1/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"  
192.168.1.30 - - [19/Sep/2026:11:09:27 +0000] "GET /poweredby.png HTTP/1.1" 200 5714 "http://192.168.1.1  
/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"  
192.168.1.30 - - [19/Sep/2026:11:09:27 +0000] "GET /favicon.ico HTTP/1.1" 404 196 "http://192.168.1.1/"  
"Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"  
^C  
[root@server.emcikunova.net ~]#
```

Рисунок 2.4: Анализ журналов error_log и access_log HTTP-сервера

Для настройки виртуального хостинга необходимо обеспечить разрешение доменных имён server.emcikunova.net и www.emcikunova.net. Перед изменением файлов DNS-зон останавливаю службу BIND командой `systemctl stop named`.

В файл прямой зоны `/var/named/master/fz/emcikunova.net` добавляю ресурсную запись `www A 192.168.1.1` (см. рис. Рисунок 2.5). В зоне уже присутствует запись `server A 192.168.1.1`, поэтому оба DNS-имени будут разрешаться в один IP-адрес веб-сервера.

Запись типа A устанавливает соответствие между именем узла и его IPv4-адресом. В результате запросы к server.emcikunova.net и www.emcikunova.net будут направляться на внутренний интерфейс виртуальной машины server с адресом 192.168.1.1.

```
1 $ORIGIN .
2 $TTL 86400 ; 1 day
3 emcikunova.net IN SOA emcikunova.net. server.emcikunova.net. (
4     2026091900 ; serial
5     86400 ; refresh (1 day)
6     3600 ; retry (1 hour)
7     604800 ; expire (1 week)
8     10800 ; minimum (3 hours)
9 )
10 NS emcikunova.net.
11 A 192.168.1.1
12 $ORIGIN emcikunova.net.
13 $TTL 1200 ; 20 minutes
14 client A 192.168.1.30
15 DHCID ( AAEBfR4Mk/22GvzzmJZs4a9bjH2cfBbBaEMQAw7BevAm
16 oOQ= ) ; 1 1 32
17 $TTL 86400 ; 1 day
18 dhcp A 192.168.1.1
19 ns A 192.168.1.1
20 server A 192.168.1.1
21 www A 192.168.1.1
22
```

Рисунок 2.5: Добавление записи www в прямую DNS-зону emcikunova.net

Далее редактирую файл обратной зоны `/var/named/master/rz/192.168.1` и добавляю для адреса `192.168.1.1` PTR-запись `www.emcikunova.net`. (см. рис. Рисунок 2.6).

PTR-записи выполняют обратное преобразование IP-адреса в DNS-имя. В зоне уже присутствуют имена `server.emcikunova.net`, `ns.emcikunova.net` и `dhcp.emcikunova.net`, связанные с адресом сервера. Добавление `www.emcikunova.net` обеспечивает наличие обратной DNS-записи и для имени виртуального веб-сервера.

После внесения изменений удаляю журнальные файлы динамического обновления DNS-зон, если они существуют, и снова запускаю DNS-сервер командой `systemctl start named`.

```
1 $ORIGIN .
2 $TTL 86400 ; 1 day
3 1.168.192.in-addr.arpa IN SOA 1.168.192.in-addr.arpa. server.emcikunova.net. (
4                               2026091900 ; serial
5                               86400 ; refresh (1 day)
6                               3600 ; retry (1 hour)
7                               604800 ; expire (1 week)
8                               10800 ; minimum (3 hours)
9                               )
10 NS 1.168.192.in-addr.arpa.
11 A 192.168.1.1
12 PTR server.emcikunova.net.
13 $ORIGIN 1.168.192.in-addr.arpa.
14 1 PTR server.emcikunova.net.
15 PTR ns.emcikunova.net.
16 PTR dhcp.emcikunova.net.
17 PTR www.emcikunova.net.
18 $TTL 1200 ; 20 minutes
19 30 PTR client.emcikunova.net.
20 DHCID ( AAEBfR4Mk/22GvzzmJZs4a9bjH2cfBbBaEMQAw7BevAm
21 oOQ= ) ; 1 1 32
```

Рисунок 2.6: Добавление PTR-записи `www.emcikunova.net` в обратную DNS-зону

Для виртуального веб-сервера `server.emcikunova.net` в каталоге `/etc/httpd/conf.d` создаю конфигурационный файл `server.emcikunova.net.conf` (см. рис. Рисунок 2.7).

В директиве `<VirtualHost *:80>` указываю, что виртуальный хост должен принимать HTTP-соединения на порту 80 на всех доступных адресах сервера. Параметр `ServerAdmin webmaster@emcikunova.net` определяет адрес администратора веб-сайта.

В `DocumentRoot` задаю каталог `/var/www/html/server.emcikunova.net`, содержимое которого Apache будет возвращать пользователю при обращении к данному виртуальному хосту. Директива `ServerName server.emcikunova.net` задаёт DNS-имя, по которому Apache выбирает эту конфигурацию на основании HTTP-заголовка `Host`.

Для виртуального хоста также создаются отдельные журналы. Ошибки записываются в `logs/server.emcikunova.net-error_log`, а обращения клиентов — в `logs/server.emcikunova.net-access_log`. Путь `logs` в конфигурации Apache соответствует системному каталогу журналов веб-сервера.



```

1 <VirtualHost *:80>
2     ServerAdmin webmaster@emcikunova.net
3     DocumentRoot /var/www/html/server.emcikunova.net
4     ServerName server.emcikunova.net
5     ErrorLog logs/server.emcikunova.net-error_log
6     CustomLog logs/server.emcikunova.net-access_log common
7 </VirtualHost>

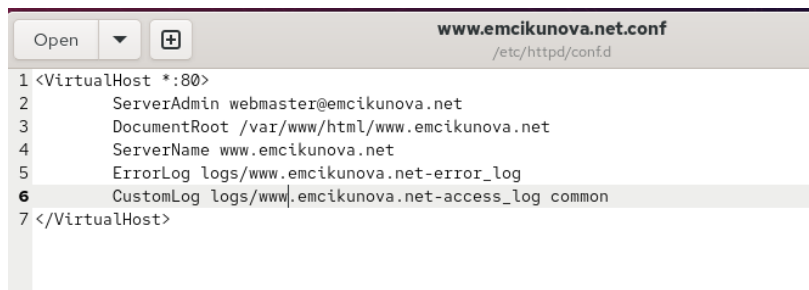
```

Рисунок 2.7: Конфигурация виртуального хоста server.emcikunova.net

Аналогично создаю файл `/etc/httpd/conf.d/www.emcikunova.net.conf` для второго виртуального хоста (см. рис. Рисунок 2.8).

Для него задаю `ServerName www.emcikunova.net`, а корневым каталогом веб-сайта назначаю `/var/www/html/www.emcikunova.net`. Отдельные файлы `www.emcikunova.net-error_log` и `www.emcikunova.net-access_log` позволяют независимо анализировать ошибки и обращения к этому веб-сайту.

Оба виртуальных хоста используют один IP-адрес 192.168.1.1 и один TCP-порт 80. Выбор нужного сайта выполняется Apache по имени, которое клиент передаёт в HTTP-заголовке `Host`. Это позволяет размещать несколько сайтов на одном сетевом интерфейсе.



```

1 <VirtualHost *:80>
2     ServerAdmin webmaster@emcikunova.net
3     DocumentRoot /var/www/html/www.emcikunova.net
4     ServerName www.emcikunova.net
5     ErrorLog logs/www.emcikunova.net-error_log
6     CustomLog logs/www.emcikunova.net-access_log common
7 </VirtualHost>

```

Рисунок 2.8: Конфигурация виртуального хоста www.emcikunova.net

В каталоге `/var/www/html` создаю подкаталог `server.emcikunova.net`, предназначенный для содержимого первого виртуального хоста. В нём создаю файл `index.html` со строкой `Welcome to the server.emcikunova.net server.` (см. рис. Рисунок 2.9).

Файл `index.html` является индексной страницей веб-сервера и автоматически возвращается Apache при обращении клиента к корневому URL виртуального хоста.

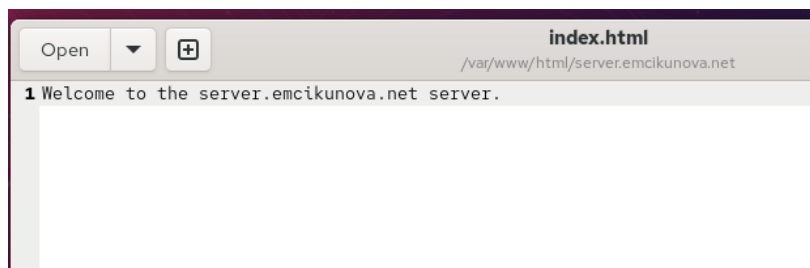


Рисунок 2.9: Тестовая страница виртуального хоста `server.emcikunova.net`

Для второго виртуального хоста создаю каталог `/var/www/html/www.emcikunova.net` и файл `index.html`. В него помещаю строку `Welcome to the www.emcikunova.net server.` (см. рис. Рисунок 2.10).

Различное содержимое индексных страниц позволяет при последующей проверке однозначно определить, какой виртуальный хост был выбран Apache при обработке запроса.

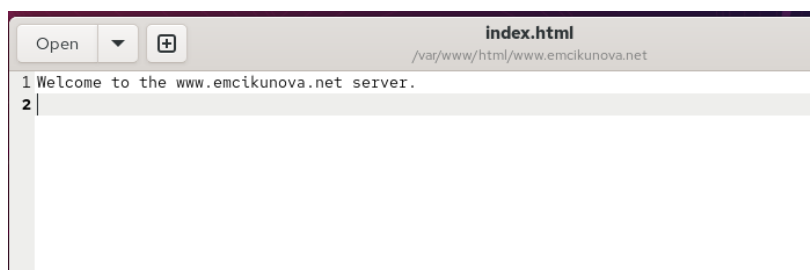


Рисунок 2.10: Тестовая страница виртуального хоста `www.emcikunova.net`

После создания каталогов и веб-страниц назначаю владельцем дерева `/var/www` системного пользователя и группу Apache командой `chown -R apache:apache /var/www/` (см. рис. Рисунок 2.11). Это обеспечивает веб-серверу необходимые права для работы с размещённым содержимым.

Далее восстанавливаю корректные контексты SELinux командами `restorecon -vR /etc`, `restorecon -vR /var/named` и `restorecon -vR /var/www`.

Команда `restorecon` назначает объектам контексты безопасности в соответствии с системной политикой SELinux. На экране видно, что для отдельных файлов действительно выполняется изменение меток.

После корректировки прав и контекстов перезапускаю Apache командой `systemctl restart httpd`. Команда завершается без сообщения об ошибке, следовательно, созданные конфигурационные файлы виртуальных хостов принимаются веб-сервером.

```
[root@server.emcikunova.net www.emcikunova.net]#  
[root@server.emcikunova.net www.emcikunova.net]# chown -R apache:apache /var/www/  
[root@server.emcikunova.net www.emcikunova.net]# restorecon -vR /etc  
Relabeled /etc/NetworkManager/system-connections/eth1.nmconnection from unconfined_u:object_r:user_tmp_t:s0 to unconfined_u:object_r:NetworkManager_etc_rw_t:s0  
Relabeled /etc/named.conf from unconfined_u:object_r:etc_t:s0 to unconfined_u:object_r:named_conf_t:s0  
[root@server.emcikunova.net www.emcikunova.net]# restorecon -vR /var/named/  
[root@server.emcikunova.net www.emcikunova.net]# restorecon -vR /var/www/  
[root@server.emcikunova.net www.emcikunova.net]# systemctl restart httpd  
[root@server.emcikunova.net www.emcikunova.net]#
```

Рисунок 2.11: Настройка прав, восстановление контекстов SELinux и перезапуск Apache

На виртуальной машине `client` проверяю первый виртуальный хост, введя в браузере адрес `http://server.emcikunova.net` (см. рис. Рисунок 2.12).

DNS-сервер разрешает имя `server.emcikunova.net` в адрес `192.168.1.1`, после чего HTTP-запрос поступает на Apache. В ответ отображается содержимое созданного ранее файла `/var/www/html/server.emcikunova.net/index.html`: `Welcome to the server.emcikunova.net server.`

Это подтверждает совместную корректную работу DNS-разрешения имён и механизма виртуальных хостов Apache.

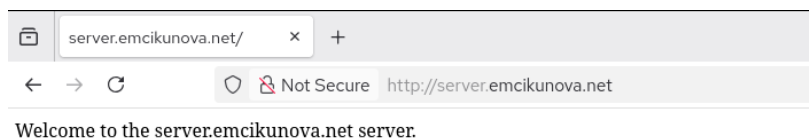


Рисунок 2.12: Проверка виртуального хоста `server.emcikunova.net` с машины `client`

Затем в браузере виртуальной машины `client` открываю адрес `http://www.emcikunova.net` (см. рис. Рисунок 2.13).

Веб-сервер возвращает строку `Welcome to the www.emcikunova.net server.`, расположенную в `/var/www/html/www.emcikunova.net/index.html`. Несмотря на использование того же IP-адреса и порта, Apache выбирает другой `DocumentRoot` на основании имени `www.emcikunova.net`.

Успешное отображение обеих тестовых страниц подтверждает корректность настройки виртуального хостинга. Пометка `Not Secure` связана с использованием HTTP без TLS и не является ошибкой конфигурации.

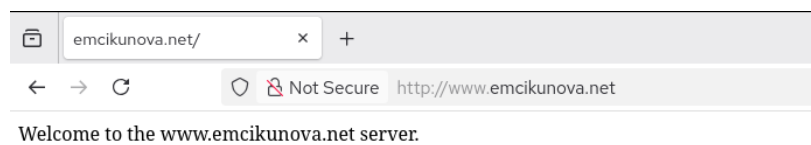


Рисунок 2.13: Проверка виртуального хоста `www.emcikunova.net` с машины `client`

Для сохранения выполненной конфигурации в проекте Vagrant перехожу в каталог `/vagrant/provision/server/` и создаю структуру каталогов для HTTP-сервера (см. рис. Рисунок 2.14).

Командами `mkdir -p /vagrant/provision/server/http/etc/httpd/conf.d` и `mkdir -p /vagrant/provision/server/http/var/www/html` создаю каталоги, повторяющие расположение конфигурации и содержимого веб-сервера внутри виртуальной машины.

После этого копирую файлы из `/etc/httpd/conf.d/` в `/vagrant/provision/server/http/etc/httpd/conf.d/` а содержимое `/var/www/html/` — в `/vagrant/provision/server/http/var/www/html/`. Благодаря этому конфигурации виртуальных хостов и тестовые страницы становятся частью проекта и могут быть автоматически восстановлены при

повторном развёртывании виртуальной машины.

Также перехожу в `/vagrant/provision/server/dns/` и копирую содержимое `/var/named/` в `/vagrant/provision/server/dns/var/named/`, сохраняя изменённые прямую и обратную DNS-зоны.

Во время копирования DNS-файлов появляются сообщения `cannot create symbolic link ...: Protocol error` для нескольких стандартных символических ссылок BIND. Они связаны с особенностями общей директории Vagrant, в которой создание символических ссылок может быть ограничено. Основные файлы зон, включая изменённые `emcikunova.net` и `192.168.1`, при этом копируются. Для полной воспроизводимости DNS-конфигурации наличие файлов, на которые указывают не созданные ссылки, следует контролировать отдельно.

В завершение создаю файл `/vagrant/provision/server/http.sh`, предназначенный для автоматизации установки и настройки HTTP-сервера.

```
[root@server.emcikunova.net www.emcikunova.net]#
[root@server.emcikunova.net www.emcikunova.net]# cd /vagrant/provision/server/
[root@server.emcikunova.net server]# mkdir -p /vagrant/provision/server/http/etc/httpd/conf.d
[root@server.emcikunova.net server]# mkdir -p /vagrant/provision/server/http/var/www/html
[root@server.emcikunova.net server]# cp -R /etc/httpd/conf.d/* /vagrant/provision/server/http/etc/httpd/
conf.d/
[root@server.emcikunova.net server]# cp -R /var/www/html/* /vagrant/provision/server/http/var/www/html/
[root@server.emcikunova.net server]# cd /vagrant/provision/server/dns/
[root@server.emcikunova.net dns]# cp -R /var/named/* /vagrant/provision/server/dns/var/named/
cp: overwrite '/vagrant/provision/server/dns/var/named/data/named.run'? y
cp: overwrite '/vagrant/provision/server/dns/var/named/dynamic/managed-keys.bind.jnl'? y
cp: overwrite '/vagrant/provision/server/dns/var/named/dynamic/managed-keys.bind'? y
cp: overwrite '/vagrant/provision/server/dns/var/named/master/fz/emcikunova.net'? y
cp: overwrite '/vagrant/provision/server/dns/var/named/master/tz/192.168.1'? y
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.ca': Protocol error
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.empty': Protocol error
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.localhost': Protocol error
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.loopback': Protocol error
[root@server.emcikunova.net dns]# cd /vagrant/provision/server/
[root@server.emcikunova.net server]# touch http.sh
[root@server.emcikunova.net server]#
```

Рисунок 2.14: Сохранение конфигурации HTTP и DNS в структуре provision

В созданный файл `http.sh` помещаю provisioning-скрипт, повторяющий основные действия, выполненные вручную (см. рис. Рисунок 2.15).

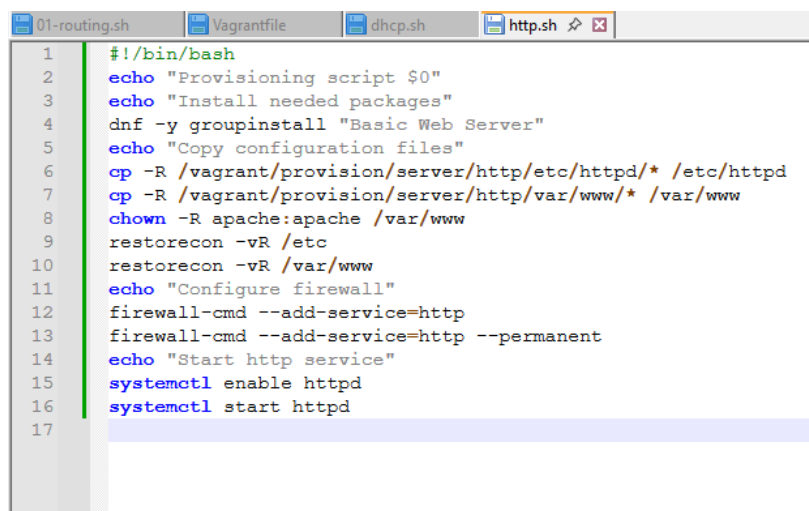
Скрипт начинается с `#!/bin/bash`, после чего выводит информационные сообщения о текущих этапах настройки. Командой `dnf -y groupinstall "Basic Web Server"` автоматически устанавливается HTTP-сервер и необходимые па-

кеты.

Затем командами `cp -R` сохранённые в проекте конфигурационные файлы Apache и содержимое веб-каталогов копируются обратно в `/etc/httpd` и `/var/www`. Команда `chown -R apache:apache /var/www` устанавливает необходимого владельца веб-контента, а `restorecon -vR /etc` и `restorecon -vR /var/www` восстанавливают контексты SELinux.

После этого скрипт разрешает HTTP в `firewalld` как для текущего сеанса, так и постоянно командами `firewall-cmd --add-service=http` и `firewall-cmd --add-service=http --permanent`.

На последнем этапе `systemctl enable httpd` включает автоматический запуск Apache, а `systemctl start httpd` запускает службу. Файлу `http.sh` назначаю право на выполнение командой `chmod +x http.sh`.



```
01-routing.sh Vagrantfile dhcp.sh http.sh
1  #!/bin/bash
2  echo "Provisioning script $0"
3  echo "Install needed packages"
4  dnf -y groupinstall "Basic Web Server"
5  echo "Copy configuration files"
6  cp -R /vagrant/provision/server/http/etc/httpd/* /etc/httpd
7  cp -R /vagrant/provision/server/http/var/www/* /var/www
8  chown -R apache:apache /var/www
9  restorecon -vR /etc
10 restorecon -vR /var/www
11 echo "Configure firewall"
12 firewall-cmd --add-service=http
13 firewall-cmd --add-service=http --permanent
14 echo "Start http service"
15 systemctl enable httpd
16 systemctl start httpd
17
```

Рисунок 2.15: Provisioning-скрипт `http.sh` для автоматической настройки HTTP-сервера

3 Вывод

В результате работы на виртуальной машине `server` установлен и настроен HTTP-сервер Apache. Для него разрешён HTTP-трафик в `firewalld`, включён автоматический запуск и проверена доступность с виртуальной машины `client`.

Настроены два виртуальных хоста: `server.emcikunova.net` и `www.emcikunova.net`. Для каждого создан отдельный каталог с веб-контентом и настроены собственные журналы. Работа виртуальных хостов успешно проверена через браузер.

Конфигурация HTTP-сервера сохранена в каталоге `provision`, а действия по установке и настройке записаны в скрипте `http.sh`, который подключён в `Vagrantfile`.

4 Контрольные вопросы

1. Через какой порт по умолчанию работает Apache?

Apache по протоколу HTTP по умолчанию работает через TCP-порт 80. Для HTTPS обычно используется порт 443.

2. Под каким пользователем запускается Apache и к какой группе относится этот пользователь?

Рабочие процессы Apache в Rocky Linux запускаются от имени пользователя apache, который входит в группу apache.

3. Где располагаются лог-файлы веб-сервера? Что можно по ним отслеживать?

Основные журналы Apache располагаются в каталоге:

`/var/log/httpd/`

Файл `access_log` содержит сведения о запросах клиентов, а `error_log` — сообщения об ошибках и работе веб-сервера.

4. Где по умолчанию содержится контент веб-серверов?

По умолчанию веб-контент Apache располагается в каталоге:

`/var/www/html/`

Для виртуальных хостов можно назначать отдельные каталоги с помощью директивы `DocumentRoot`.

5. Каким образом реализуется виртуальный хостинг? Что он даёт?

Виртуальный хостинг в Apache реализуется с помощью блоков `<VirtualHost>`. Для каждого сайта задаются `ServerName`, `DocumentRoot` и другие параметры.

Это позволяет размещать несколько сайтов на одном сервере и одном IP-адресе. Apache выбирает нужный сайт по доменному имени, переданному

клиентом в HTTP-запросе.